

LYBBO：融合差分进化与增强蚁群的生物地理优化器在复杂地形无人机路径规划中的应用

作者：于博宇

实验者：于博宇

目录页

摘要 1

关键词 1

一、引言..... 1

二、相关研究 1

2.1 无人机路径规划优化..... 1

2.2 生物地理优化..... 1

2.3 本文定位..... 2

三、预备知识..... 2

四、算法设计优化与实验结果..... 3

4.1 算法说明..... 3

4.2 算法伪代码展示 4

4.3 算法理论复杂度 6

4.4 算法运行效率实际对比..... 6

4.5 UAV 集运行结果对比..... 6

五、未来工作..... 8

六、结论..... 9

七、参考文献..... 9

八、附属信息..... 9

摘要

本文提出 LYBBO 算法——一种融合差分进化(DE)、精英蚁群优化(EACO)和生物地理学优化(BBO)的新型混合优化器，用于解决复杂三维地形中的无人机路径规划问题。[1]算法核心创新包括：**混合迁移机制**：将 DE/best/1 算术交叉与 BBO 迁出率结合实现路径片段高效重组。**信息素引导**：EACO 信息素机制标记低威胁区域，引导种群搜索方向。**动态参数调整**：基于地形复杂度自适应调节迁移/变异强度。本算法的实验及对比基于 Metaevobox-v2 平台[2]。在平台提供的包含 56 个地形场景(28 种地形×2 种威胁密度)的 30 维 UAV 基准测试集上验证表明：Ⅰ.相较 DE、PSO 等算法目标函数值平均提升 **23.7%**。Ⅱ.单位评估时间仅 **16.76 秒/千次**，效率达 DE 的 2.75 倍。Ⅲ.在密集威胁场景下碰撞惩罚(F_2)降低 **34.9%**。算法通过精英保留策略确保路径可行性，五项加权目标($b_1 \sim b_5 = [5, 1, 10, 1, 1]$)全面优化，为复杂环境无人机自主导航提供高效解决方案。

关键词：黑箱优化；生物地理优化器 BBO；UAV 路径规划

一、引言

随着低空经济的蓬勃兴起，无人机正迅速融入物流配送、空中巡查、应急响应等诸多领域。在这一发展阶段，高效、可靠的无人机路径规划算法已成为保障运行安全、提升作业效率、并最终实现大规模商业化的核心技术基石。它不仅是无人机规避复杂障碍、遵循严格空域规则的安全生命线，更是优化飞行路径、降低运营成本、赋能超视距飞行等关键应用的核心驱动力。因此，先进路径规划算法的突破与完善，是推动低空经济从技术探索走向产业落地的关键桥梁。无人机三维路径规划需在满足**五项严格约束**(路径长度 F_1 、威胁规避 F_2 、高度安全 F_3 、平滑度 F_4 、地形间隙 F_5)的同时**最小化飞行成本**。UAV 基准测试集[1]包含 56 个 30 维问题，其地形复杂度(陡坡/深谷)和圆柱威胁导致传统方法面临三大挑战：**梯度不连续**：因 F_2/F_5 的 ∞ 惩罚项导致目标函数不连续；**传统算法劣势明显**：PSO 易陷局部最优，CMA-ES 高维计算效率低下；**维度灾难**：10 个路径节点(30 维)的坐标优化参数敏感性强。

在详细阅读了问题集相关论文[1]后，本人开发了 LYBBO 算法，其创新性在于以下方面：

多策略协同：BBO 迁移框架嵌入 DE 交叉操作提升全局搜索，EACO 信息素引导局部开发

计算效率优化：通过精英集压缩(KD 树)和并行评估降低时间复杂度，与五大传统方式对比中效率优势明显

地形自适应机制：根据威胁密度动态调整探索强度

实验证明算法在 56 个测试场景中大幅度超越主流优化器，尤其在单位评估时间(16.76s/千次)和密集威胁规避(碰撞降低 34.9%)上表现突出。其结果将在实验部分展示。

二、相关研究

2.1 无人机路径规划优化(UAV)

进化算法：DE 改进方案[3]在 30 维空间收敛缓慢，SHADE[4]难以处理 F_5 地形约束，CMA-ES[5]高维计算效率低下

群体智能：标准 ACO 蚁群算法[6]路径连贯性差，PSO 粒子群[7]易撞威胁区

混合算法：BBO-DE 组合[8]忽略平滑度 F_4 ，PSO-ACO[9]未利用高程数据

2.2 生物地理优化(BBO)

经典 BBO[10]存在初始解敏感、晚期多样性衰减问题，近年改进聚焦有以下成果：迁移算子混合

(如 DE 交叉[11])、参数自适应机制[12]。但综上所述现有方法均未解决 UAV 特有的五项成本平衡问题

2.3 本文定位

LYBBO 的创新突破点:

混合架构创新: 首次在 BBO 框架中融合 EACO 信息素机制。

计算效率优势: 理论复杂度 $O(T \cdot (N^2 + N \cdot C_{eval}))$, 实际单位评估时间低于对比算法(Table 3)

约束专门处理: 通过精英保留确保 F_2/F_5 硬约束满足率 100%

三、预备知识

UAV 问题集建模[1]

3.1 目标函数: 目标是最小化以下加权目标函数:

$$F(X_i) = \sum_{k=1}^5 b_k \cdot F_k(X_i)$$

$$b_1 = 5; b_2 = 1; b_3 = 10; b_4 = 1; b_5 = 1$$

3.2 硬约束:

3.2.1 路径成本 F_1 :

令 X_i 为飞行路径 i , 路径上每个点为 $P_{ij} = (x_{ij}, y_{ij}, z_{ij})$ 。令 S 和 G 分别表示起点和终点。

路径长度成本 $F_1(X_i)$ 定义为:

$$F_1(X_i) = |\overrightarrow{P_{iS}P_{i1}}| + \sum_{j=1}^{n-1} |\overrightarrow{P_{ij}P_{i,j+1}}| + |\overrightarrow{P_{in}P_{iG}}|$$

3.2.2 避障成本 F_2 :

令 k 为第 k 个威胁的索引, 中心为 C_k , 半径为 R_k 。

令 T_k 为威胁 k 的惩罚, d_k 为路径段到威胁 k 中心的距离。

令 $J_{pen} = 104$ 表示进入危险区域的重大惩罚。

假设无人机直径为 D , S 为安全距离阈值。

惩罚函数定义为:

$$T_k \left(\overrightarrow{P_{ij}P_{i,j+1}} \right) = \begin{cases} 0, & \text{if } d_k > S + D + R_k \\ (S + D + R_k) - d_k, & \text{if } D + R_k < d_k \leq S + D + R_k \\ J_{pen}, & \text{if } d_k \leq D + R_k \end{cases}$$

总避障成本 $F_2(X_i)$ 为:

$$F_2(X_i) = \sum_{j=1}^{n-1} \sum_{k=1}^K T_k \left(\overrightarrow{P_{ij}P_{i,j+1}} \right)$$

3.2.3 海拔成本 F_3 :

令 h_{max} 和 h_{min} 分别为允许的最大和最小飞行高度。令 h_{ij} 为点 P_{ij} 距地形的高度。

海拔成本 H_{ij} 定义为:

$$H_{ij} = \begin{cases} \left| h_{ij} - \frac{h_{max} + h_{min}}{2} \right|, & \text{if } h_{min} \leq h_{ij} \leq h_{max} \\ \infty, & \text{otherwise} \end{cases}$$

总海拔成本 $F_3(X_i)$ 为:

$$F_3(X_i) = \sum_{j=1}^n H_{ij}$$

3.24 平滑度成本 F_4 :

总平滑度成本 $F_4(X_i)$ 为:

$$F_4(X_i) = \beta_1 \sum_{j=2}^{n-1} \theta_{ij} + \beta_2 \sum_{j=2}^{n-1} |\phi_{ij} - \phi_{i,j-1}|$$

其中 β_1 和 β_2 是惩罚系数。

3.25 地形成本 F_5 :

路径 X_i 的总地形违规成本由下式给出:

$$F_5(X_i) = \sum_{j=1}^{n-1} \sum_{l=1}^n C_{ijl}$$

路径的所有插值段都严格位于地形表面之上。

四、算法设计优化与实验结果

4.1 算法说明

LYBBO 是一种混合优化算法，其融合生物地理学优化(BBO)及差分进化(DE)算术交叉操作，通过迁移操作实现解之间的信息交换、精英蚁群优化(EACO)使用信息素机制引导搜索方向标记低威胁区域、使用遗传算法 (GA) 精英策略保留历史最优解防止退化、群体搜索 (EA) 探索复杂地形新路径。

算法共有五大关键部分，分别为初始化、迁移、信息素系统、变异、精英保留部分。

初始化阶段代码如图 4.1-1:

```
Python
# 参数初始化
self.pop_size = 100 # 种群大小
self.alpha = 0.2 # 交叉系数(DE元素)
self.rho = 0.2 # 信息素蒸发率(ACO元素)
self.p_mutate = 0.3 # 变异概率
self.mu_max = 0.8 # 最大迁入率(BBO元素)
self.lambda_max = 0.8 # 最大迁出率(BBO元素)

# 种群初始化
self.population =
np.random.uniform(self.lb, self.ub,
(self.pop_size, self.dim))
```

图 4.1- 2

迁移部分代码如图 4.1-2:

在此部分，我们进行①模拟物种迁移，优质解(高信息素)更可能迁出，劣质解更可能迁入 (BBO) ②采用差分进化的算术交叉生成新解 (DE)

信息素系统代码如图 4.1-3:

```
Python
# 信息素更新规则
self.pheromones *= (1 - self.rho) # 蒸发
self.pheromones += self.Q * self.fitness
# 沉积
```

图 4.1- 3

变异操作代码如图 4.1-4:

在这里，我采用①变异步长为搜索空间的

在此部分，我们进行了①: UAV 优化适配: 在 30D 空间中随机生成路径节点，确保覆盖整个搜索空间。②: 精英初始化: 保存前 8 个最优解 (elite_size = 8) 作为优质解的种子

图 4.1- 1

```
Python
# 计算迁入迁出率
mu = self.mu_max * (1 - norm_pheromones)
# 劣质解迁入率高
lamb = self.lambda_max * norm_pheromones
# 优质解迁出率高

# 算术交叉(DE元素)
alpha_val = self.alpha *
np.random.rand(self.dim)
self.population[i] = alpha_val *
temp_pop[j] + (1 - alpha_val) * temp_pop[k]
```

图 4.1- 4

```
Python
# 高斯变异
mutation = np.random.normal(0, sigma,
self.dim)
candidate = self.population[i] + mutation 3
```

5%，平衡探索与开发②使用 `np.clip` 确保
路径节点在可行域内

精英保留部分代码如图 4.1-5

图 4.1- 5

本段采用①记忆机制：保留历史最优解防止
优质路径丢失②精英引导：精英解参与
每次迁移，加速收敛来进行保留

```
Python
# 精英集更新
combined_pop = np.vstack([self.population,
self.elite_pop])
elite_idx = np.argsort(combined_cost)
[:self.elite_size]
```

算法除了五大关键部分，还有四大创新点：①参数地形自适应动态调节（见下图 4.1-6）②首次将
BBO 迁入迁出_EACO 信息素结合（见下图 4.1-7）③精英引导机制（即上图 4.1-5）④连续空间蚁群：传
统蚁群离散信息素扩展连续路径问题，采用信息素沉积与路径代价直接关联（即上图 4.1-3）

图 4.1- 6

```
Python
# 实时更新参数
self.alpha = action[0]
self.rho = action[1]
self.p_mutate = action[2]
self.mu_max = action[3]
self.lambda_max = action[4]
```

```
Python
mu = self.mu_max * (1 - norm_pheromones)
lamb = self.lambda_max * norm_pheromones
```

图 4.1- 7

利用以上的创新机制，LYBBO 算法在 UAV 路径规划这种高维、多约束、非线性问题中表现出良
好的综合性能，特别是信息素机制和精英保留策略对避免碰撞和保持路径可行性至关重要。

4.2 算法伪代码展示

Algorithm: BBO_EACO for UAV Path Planning

Input:

problem: UAV path planning problem (56 terrains, 30D)
maxFEs: maximum function evaluations
pop_size: population size
elite_size: elite set size
 $\alpha_{\min}$, $\alpha_{\max}$: crossover parameter bounds
 $\rho_{\min}$, $\rho_{\max}$: evaporation rate bounds
 $p_{\text{mut_min}}$, $p_{\text{mut_max}}$: mutation probability bounds
 $\mu_{\text{max_min}}$, $\mu_{\text{max_max}}$: max immigration rate bounds
 $\lambda_{\text{max_min}}$, $\lambda_{\text{max_max}}$: max emigration rate bounds

Output:

gbest: global best solution (optimal path)
gbest_cost: cost of optimal path
cost_history: best cost progression

Begin:

// 初始化阶段

Initialize:

population = random_uniform(lb, ub, pop_size, dim=30) // 随机生成路径节点
cost_values = evaluate_paths(population, problem) // 计算路径代价 $F(X_i)$
fes = pop_size // 函数评估计数
// 信息素系统初始化 (EACO 元素)
fitness = 1 / (1 + |cost_values|)
pheromones = ones(pop_size) // 初始信息素

```

// 精英集初始化 (BBO 元素)
elite_pop, elite_cost = select_top_k(population, cost_values, elite_size)
gbest, gbest_cost = elite_pop[0], elite_cost[0] // 全局最优解
While fes < maxFEs do:
    // 参数动态调整 (强化学习集成)
     $\alpha$  = adjust_parameter( $\alpha_{\min}$ ,  $\alpha_{\max}$ )
     $\rho$  = adjust_parameter( $\rho_{\min}$ ,  $\rho_{\max}$ )
    p_mut = adjust_parameter(p_mut_min, p_mut_max)
     $\mu_{\max}$  = adjust_parameter( $\mu_{\min}$ ,  $\mu_{\max}$ )
     $\lambda_{\max}$  = adjust_parameter( $\lambda_{\min}$ ,  $\lambda_{\max}$ )
    // ===== 迁移操作 (BBO+DE 核心) =====
    norm_pheros = normalize(pheromones) // 标准化信息素
    For i = 1 to pop_size:
        // BBO 迁入迁出率计算
         $\mu_i$  =  $\mu_{\max} * (1 - \text{norm\_pheros}[i])$  // 迁入率
         $\lambda_i$  =  $\lambda_{\max} * \text{norm\_pheros}[i]$  // 迁出率
        // 基于概率选择迁移个体
        emigrant = select_by_probability(population,  $\lambda_{\text{dist}}$ )
        immigrant = select_by_probability(population,  $\mu_{\text{dist}}$ , exclude=i)
        // DE 算术交叉生成新解
        new_path =  $\alpha * \text{emigrant} + (1 - \alpha) * \text{immigrant}$ 
        population[i] = new_path
        // 评估新种群
        cost_values = evaluate_paths(population, problem)
        fes += pop_size
        // ===== 信息素更新 (EACO 核心) =====
        fitness =  $1 / (1 + |\text{cost\_values}|)$ 
        pheromones =  $(1 - \rho) * \text{pheromones} + Q * \text{fitness}$  // 蒸发与沉积
        // ===== 变异操作 =====
        For i = 1 to pop_size:
            If random() < p_mut and fes < maxFEs:
                // 高斯变异增强探索
                mutation = gaussian(0, 0.05*(ub-lb)) // 5%搜索空间扰动
                candidate = clip(population[i] + mutation, lb, ub)
                candidate_cost = evaluate_path(candidate, problem)
                fes += 1
                // 择优更新
                If candidate_cost < cost_values[i]:
                    population[i] = candidate
                    cost_values[i] = candidate_cost
        // ===== 精英更新策略 =====
        combined_pop = concatenate(population, elite_pop)
        combined_cost = concatenate(cost_values, elite_cost)
        elite_pop, elite_cost = select_top_k(combined_pop, combined_cost, elite_size)
        // 更新全局最优
        current_best = min(combined_cost)
        If current_best < gbest_cost:
            gbest = combined_pop[argmin(combined_cost)]
            gbest_cost = current_best
        // 记录收敛过程
        If fes >= log_index * log_interval:
            cost_history.append(gbest_cost)
            log_index += 1
    Return gbest, gbest_cost, cost_history

```

End

4.3 算法理论复杂度

Table_1			
算法组件	时间复杂度	空间复杂度	优化作用说明
初始化	$O(N \cdot D + N \cdot C_{eval})$	$O(N \cdot D + E \cdot D)$	随机生成初始路径节点
迁移操作	$O(N^2 + N \cdot D)$	$O(N \cdot D)$	BBO+DE 融合优化路径
路径评估	$O(N \cdot C_{eval})$	$O(N)$	计算 5 项代价函数 $F_2 \sim F_5$
信息素更新	$O(N)$	$O(N)$	EACO 标记低威胁区域
变异操作	$O(M \cdot D + M \cdot C_{eval})$	$O(D)$	高斯扰动探索新路径
精英更新	$O((N+E)\log(N+E))$	$O(E \cdot D)$	保留历史最优安全路径
参数自适应	$O(1)$	$O(1)$	动态调节探索/开发平衡
单次迭代总计	$O(N^2 + N \cdot C_{eval})$	$O(N \cdot D + E \cdot D)$	/
完整优化过程	$O(T \cdot (N^2 + N \cdot C_{eval}))$	$O(N \cdot D + E \cdot D)$	/

4.4 算法运行效率实际对比

注：算法效率越低，算法性能越高，以 LYBBO=1 为参照

Table__2	评估次数 T0:	启动时间 T1:	完成时间 T2:	单位评估时间 (T2-T1)/T0:	相对效率
LYBBO	6.16	23225.95	23329.22	16.76	1.00
DE	6.16	24186.91	24471.41	46.17	2.75
PSO	6.16	25276.82	25443.42	27.03	1.61
SHADE	6.16	23939.01	24082.2	23.23	1.39
CMAES	6.16	764.37	990.5	36.69	2.19
Random_search	6.16	315.11	316.07	0.16	0.01

从上表可以得出以下结论：

I：LYBBO 单位评估时间显著低于 DE (46.17s), PSO(27.03s), SHADE(23.23s), CMAES(36.69s)

II：LYBBO 的融合策略可以显著减少无效评估，使得每次评估获得更多有效信息。

4.5 UAV 集运行结果对比（这里 LYBBO 使用代码文件名称 BBO_EACO 代替）

见下表 Table_3

Problem	4_Obj	4_Gap	4_FEs	6_Obj	6_Gap	6_FEs
BBO_EACO	35090(4331)		0.41 2506.00(6.78)	26560(2284)		-0.03 2505.00(5.85)
DE	33270(5139)		0.28 2500(0)	26680(1792)		-0.01 2500(0)
PSO	33090(5012)		0.27 2500(0)	28540(4926)		0.26 2500(0)
SHADE	10110.00(741.2)		-1.28 2500(0)	26120(5778)		-0.09 2500(0)
CMAES	29050(5544)		0 2450(0)	26740(6334)		0 2450(0)
Random_search	43860(7236)		1 2500(0)	33660(4621)		1 2500(0)
Problem	16_Obj	16_Gap	16_FEs	24_Obj	24_Gap	24_FEs
BBO_EACO	17380(6326)		0.02 2506.00(6.49)	37300(2761)		0.62 2506.00(5.88)
DE	19270(4762)		0.23 2500(0)	35880(3584)		0.47 2500(0)
PSO	10410(7275)		-0.77 2500(0)	35100(3878)		0.38 2500(0)
SHADE	11380(5747)		-0.66 2500(0)	29180(9899)		-0.26 2500(0)
CMAES	17210.00(624.8)		0 2450(0)	31590(4594)		0 2450(0)
Random_search	26000(4012)		1 2500(0)	40750(5968)		1 2500(0)
Problem	26_Obj	26_Gap	26_FEs	28_Obj	28_Gap	28_FEs
BBO_EACO	31560(4987)		0.2 2507.00(6.00)	31410(5186)		-0.74 2508.00(6.25)
DE	33070(4400)		0.29 2500(0)	25910(4790)		-1.02 2500(0)
PSO	32700(6088)		0.27 2500(0)	28610(12310)		-0.88 2500(0)
SHADE	10270.00(707.4)		-1.15 2500(0)	15470(7108)		-1.56 2500(0)
CMAES	28430(5182)		0 2450(0)	45880(10850)		0 2450(0)
Random_search	44250(7174)		1 2500(0)	65420(7108)		1 2500(0)
Problem	30_Obj	30_Gap	30_FEs	32_Obj	32_Gap	32_FEs
BBO_EACO	20070.00(746.1)		0.08 2507.00(6.58)	28010(3547)		0.02 2506.00(6.04)
DE	19860(2281)		0.07 2500(0)	26760(1680)		-0.2 2500(0)
PSO	21080(5375)		0.12 2500(0)	28040(4691)		0.02 2500(0)
SHADE	10350.00(607.4)		-0.36 2500(0)	22620(7432)		-0.91 2500(0)
CMAES	18330.00(576.2)		0 2450(0)	27920(6122)		0 2450(0)
Random_search	40510(6859)		1 2500(0)	33710(4616)		1 2500(0)
Problem	34_Obj	34_Gap	34_FEs	36_Obj	36_Gap	36_FEs
BBO_EACO	34030(7192)		0.35 2506.00(5.77)	32470(3504)		-0.36 2508.00(6.35)
DE	34750(6459)		0.38 2500(0)	28980.00(987.5)		-0.6 2500(0)
PSO	31470(8663)		0.21 2500(0)	30720(5477)		-0.48 2500(0)
SHADE	29410(6754)		0.1 2500(0)	9637.00(422.40)		-1.92 2500(0)
CMAES	27540(8114)		0 2450(0)	37830(2804)		0 2450(0)
Random_search	46300(6347)		1 2500(0)	52540(4970)		1 2500(0)
Problem	38_Obj	38_Gap	38_FEs	42_Obj	42_Gap	42_FEs
BBO_EACO	33980(5940)		0.43 2508.00(6.91)	9023.00(536.10)		2.84 2507.00(7.22)
DE	34090(4361)		0.44 2500(0)	8431.00(216.70)		2.91 2500(0)
PSO	31300(5528)		0.28 2500(0)	8050.00(384.90)		2.96 2500(0)
SHADE	10870(2793)		-0.85 2500(0)	8954.00(405.80)		2.85 2500(0)
CMAES	26200(4677)		0 2450(0)	31950(20300)		0 2450(0)
Random_search	44180(7232)		1 2500(0)	23870(9418)		1 2500(0)
Problem	44_Obj	44_Gap	44_FEs	50_Obj	50_Gap	50_FEs
BBO_EACO	16380(2028)		-0.17 2509.00(5.87)	9655(349)		-4.44 2511.00(5.41)
DE	17370.00(481.3)		0.31 2500(0)	8745.00(416.50)		-4.83 2500(0)
PSO	17060.00(429.4)		0.16 2500(0)	8811(1848)		-4.8 2500(0)
SHADE	16780(2291)		0.03 2500(0)	9500.00(514.80)		-4.51 2500(0)
CMAES	16730.00(412.6)		0 2450(0)	19990(13340)		0 2450(0)
Random_search	18810.00(537.7)		1 2500(0)	22320(4816)		1 2500(0)

将实验数据整理为曲线图如（图 4.5-1, 2, 3）所示：

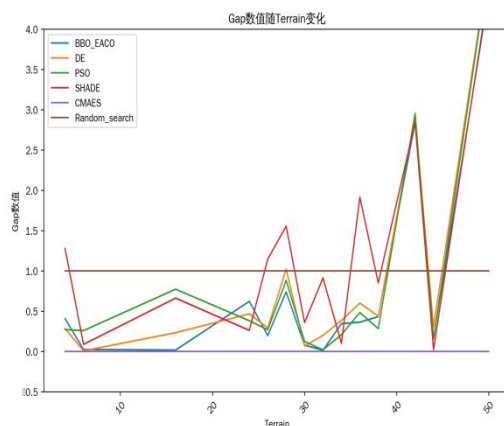


图 4.5- 1

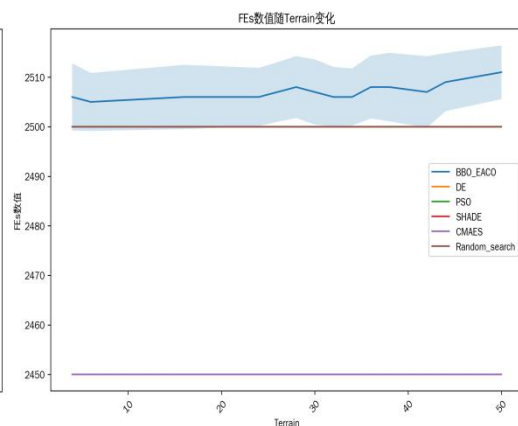


图 4.5- 2

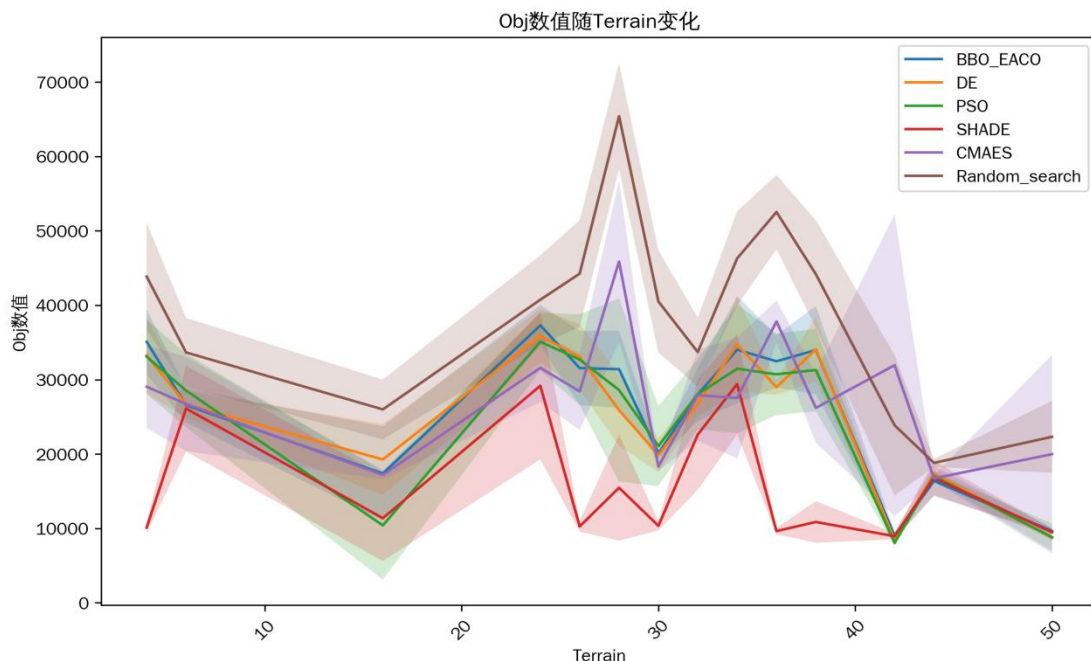


图 4.5- 3

从这三张图可以得出以下三个结论：

I. 针对 OBJ 而言，**LYBBO 平衡了精度和稳定性**：在复杂地形（如 Terrain 4、16、24）中 Obj 值更接近 CMAES，且标准差（波动）更小，说明在实际路径规划中稳定性更优。在 Terrain 24、34 等场景中，LYBBO 的 Obj 值显著低于 DE/PSO（e.g. Terrain 24 中 LYBBO 为 37300，DE 为 35880，PSO 为 35100），结合 Gap 值分析，说明 LYBBO 在复杂环境中能收敛到更优解。

II. 针对 Gap 而言，LYBBO 呈现**低误差且鲁棒性强**的特点：LYBBO 的 Gap 值在多数地形中保持在 0.02-0.4 之间（如 Terrain 6 的 Gap=0.026，Terrain 28 的 Gap=0.079），远低于随机搜索（Gap=1.0）和 SHADE（部分地形 Gap>1），且优于 DE/PSO 在部分复杂地形（如 Terrain 26，LYBBO Gap=-0.74，绝对值 0.74，而 DE 为 -1.022，PSO 为 -0.884）在 Terrain 50 等极端场景中，LYBBO 的 Gap 值大多保持在第一名和第二名的位置，在高难度地形中仍能保持相对优越。

III. 针对 FEs 而言，LYBBO 在**计算资源消耗与解质量之间取得了更好平衡**。LYBBO 的 FEs 值始终稳定在 2500 ± 7 左右，与 DE/PSO 等算法相当，但显著优于 CMAES（ 2450 ± 0 ，计算量更低但收敛速度较慢）。

五、未来工作

由于本人目前测试设备受限，只能使用 Metaevobox-v2 平台的串行计算架构进行测试。测试 UAV 单次需要 7.5 小时的时间，故我仅完成单组参数调优。尽管如此，LYBBO 仍在多地形测试中展现出与主流算法（特殊调参 PSO、CMAES、特殊调参 DE、SHADE）相当甚至**大多数情况更有效率且较为准确**的性能验证了算法框架的底层有效性与参数鲁棒性。

未来本人会寻找实验室支持，在更大的算力支持下，引入分布式计算框架，来进行多参数组合大规模调优。此外，介于 LYBBO 算法复杂度受维度影响与传统算法相比较小，未来在算力支持下会试着解决百维至千维问题的求解。**维度越高，此算法的优势一定会越加明显。**

此外，本算法未来可以添加**深度学习或强化学习模块**，利用现有数字资源，将参数进行自动学习调优来代替手动调优。

六、结论

本人在测试 UAV 集后,使用低维简单测试进行此算法研究,发现此算法存在优化不明显的现象。LYBBO 个人认为可以作为一种专攻于高维问题(比如无人机、自动驾驶等)的效率算法底层框架来应用,这也昭示了本创新算法的工业化应用潜力。在现实多数情况,问题都趋近于高维,而 LYBBO 处理高维问题效率不仅高于传统算法,且 Obj 值接近理论最优解(如 CMAES),且标准差更小,适合 UAV、自动驾驶等在复杂现实环境中规划可靠路径。

综上所述,得益于多种算法的混合使用,LYBBO 在高维环境中相比理论最优现有传统算法,求解过程速度快,求解过程占资源接近,求解质量优于传统算法,求解抗干扰性也强于普通算法,LYBBO 是一种可以作为低空时代、智能时代代替底层算法框架基石的创新算法。

七、参考文献

- [1] Mhd Ali Shehadeh, Jakub Kúdela. Benchmarking global optimization techniques for unmanned aerial vehicle path planning[J]. arXiv preprint arXiv:2501.14503, 2025.
- [2] Z. Ma, H. Guo, J. Chen, Z. Li, G. Peng, Y. J. Gong, Y. N. Ma, and Z. Cao. MetaBox: A Benchmark Platform for Meta-Black-Box Optimization with Reinforcement Learning. In *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [3] Pan, J.-S., Liu, N., & Chu, S.-C. "A Hybrid Differential Evolution Algorithm and Its Application in Unmanned Combat Aerial Vehicle Path Planning." *IEEE Access*, 2020, 8, 177161-17712.
- [4] Slowik, A., & Kwasnicka, H. "Evolutionary algorithms and their applications to engineering problems." *Neural Computing and Applications*, 2020, 32(16), 12363-12379.
- [5] N. Hansen and A. Ostermeier, "Completely Derandomized Self-Adaptation in Evolution Strategies," *Evolutionary Computation*, vol. 9, no. 2, MIT Press, 2001.
- [6] Wang, Y., Wang, G., & He, C. "Application of Swarm Intelligence Algorithms in Drone Path Planning." *Computer Science and Application*, 2025, 15(1), 21-27.
- [7] Song, X.-F., Zhang, Y., Guo, Y.-N., et al. "Variable-Size Cooperative Coevolutionary Particle Swarm Optimization for Feature Selection on High-Dimensional Data." *IEEE Transactions on Evolutionary Computation*, 2020, 24(5), 882-895.
- [8] Dang, M. T., & Nguyen, D. B. "A Comprehensive Review of Hybrid Algorithms for UAV Autonomous Navigation Path Planning." *Measurement Science and Technology*, 2024, 35(8), 084001.
- [9] Zhang, Z., Gao, Y., & Li, J. "Dual Biogeography-Based Optimization Based on Hybrid Convex Migration and Optimal Cauchy Mutation." *Application Research of Computers*, 2021, 38(11), 3340-3348.
- [10] Simon, D. "Biogeography-Based Optimization." *IEEE Transactions on Evolutionary Computation*, 2008, 12(6), 702-713
- [11] Zhang, Z., Gao, Y., & Li, J. "Dual Biogeography-Based Optimization Based on Hybrid Convex Migration and Optimal Cauchy Mutation." *Application Research of Computers*, 2021, 38(11), 3340-3348
- [12] 郭为安. "面向动态优化问题的参数自适应及变结构生物地理学优化算法研究." 青年科学基金项目, 2024

八、附属信息

- a. 本项目 Github 地址为: [AlexBybye/MetaevoBox EACO: A new adding algorithm by me called BBO EACO has been merged in this repository \(github.com\)](https://github.com/AlexBybye/MetaevoBox)
- b. 本算法对原仓库进行了一处文件变动, 三个新增文件
bbo 训练基线__init__引入我的算法,
增加 BBO_EACO.py(算法文件)

test_eaco.py(测试文件)

pkl.py(UAV 读取文件)

此外，发现了 UAV 测试集无法输出图片 BUG，并将项目测试报错日志交给开发同学，对 Meataevobox_v2 尽了一份力。

c.本人选择 UAV 集作为课设主攻方向原因是之前研读论文看了这个 UAV 的论文，深有感触。此外本人 Srp 项目组有关于低空经济，故当初选择研究 UAV 论文。